

HERALD COLLEGE KATHMANDU

Week 1

Introduction to Docker

Tutorial 2 hrs · Workshop 2.5 hrs · Phase 1, Prerequisite Course

This week you run containers. You do not build them. Building images is Week 3. The goal here is the mental model: what a container is, and how a program inside one becomes reachable from outside.

By the end of this week you can

- say what a container is, and why it beats a virtual machine for most work;
- tell an *image* from a *container*;
- run any public image from Docker Hub;
- explain why a container is unreachable by default — and publish a port to fix it;
- start, inspect, and stop containers; and diagnose the four common failures.

1 Why Containers

Software that works on your machine fails on the server, because the two machines differ — different library versions, different runtimes, different configuration. Virtual machines solved this by shipping a whole operating system with the application. It works, but costs gigabytes and takes a minute to start.

A container ships only the application and its dependencies, and **shares the host's kernel** instead of carrying its own operating system. Same isolation, a fraction of the cost.

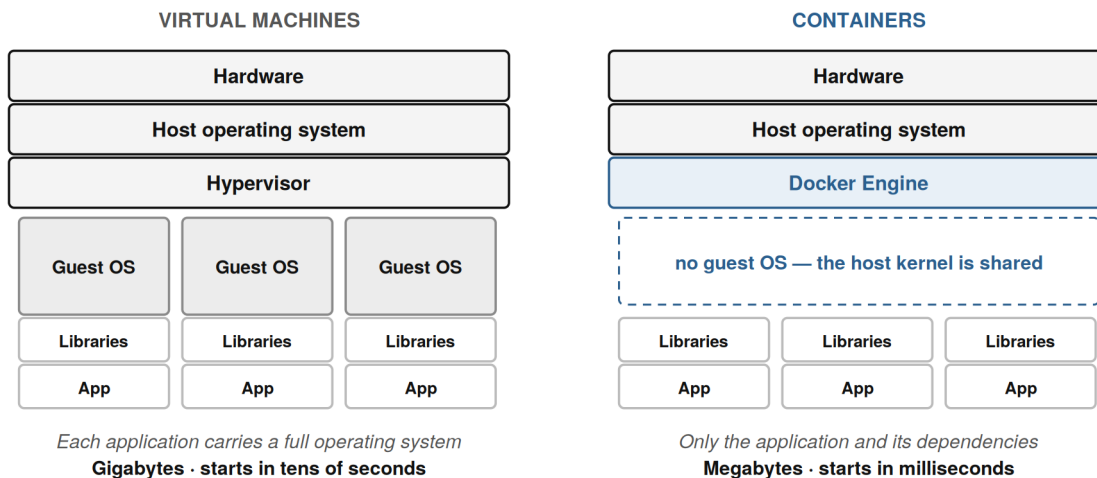


Figure 1. A virtual machine carries a full guest OS per application. A container does not.

2 Images and Containers

An **image** is a read-only template — application, runtime, libraries, config, frozen at build time. It does not run. A **container** is a running instance created from an image.

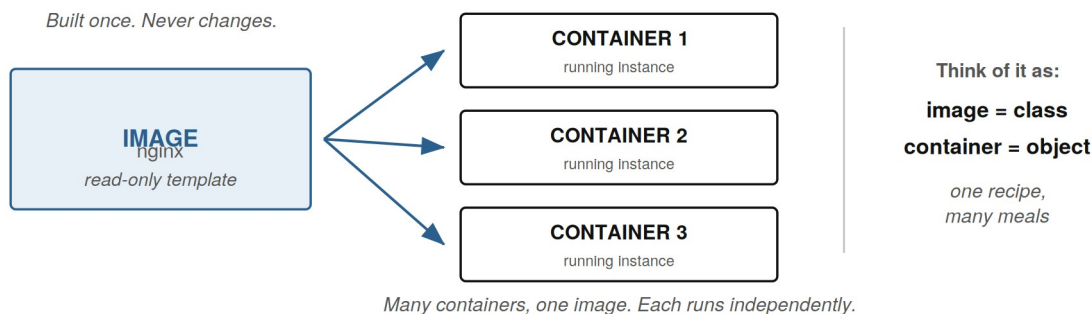


Figure 2. One image; many independent containers.

A **registry** stores and distributes images. Docker Hub is the default one; if you run an image you do not have, Docker fetches it from there automatically. Prefer *official* images — all images in this module are official.

The images we use

Image	What it gives you
nginx	A web server. Serves a page you can open in a browser.
python:3.12-slim	A Python environment, without installing Python on your machine.
node:20-slim	A Node.js environment, likewise.

3 About nginx

nginx (“engine-ex”) appears throughout this week, so it is worth thirty seconds of introduction.

- **It is a web server.** It listens on a network port and answers HTTP requests, usually by returning a file — an HTML page, an image, a stylesheet.
- **It is built for concurrency.** Older servers gave each connected client its own process and collapsed under load. nginx uses a few worker processes to handle thousands of connections, which is why it became one of the most-deployed servers in the world.
- **We use it because you can see it.** Start the container, open a browser, get a page. The effect of each command is immediately visible.
- **It returns in Week 5,** where you will put it in front of several application instances to spread traffic between them.

The nginx image listens on port 80. Remember this — it matters in the next section.

4 Setup

Install Docker Desktop (macOS, Windows) or Docker Engine (Linux) from `docs.docker.com/get-started/get-docker/`. On Windows it needs WSL2, which you already have. Then verify:

```
docker --version
docker run hello-world
```

If the second command prints a greeting, you are ready. If it says it cannot connect to the Docker daemon, Docker Desktop is not running — start it and wait.

5 Port Publishing

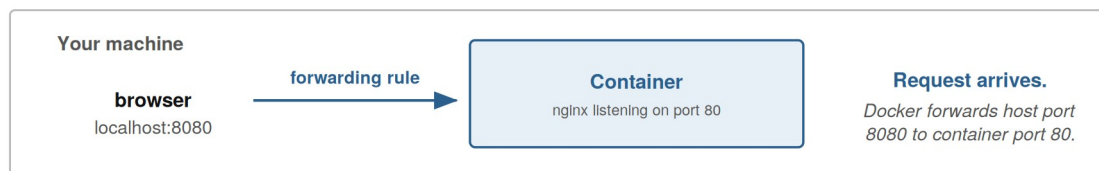
This is the one concept that trips everybody up in week one. A container has its own network. A server listening on port 80 *inside* a container is not listening on port 80 of your machine. So by default your browser cannot reach it — nothing is broken, there is simply no route.

To create that route, publish a port with `-p` when you start the container.

WITHOUT `-p`



WITH `-p 8080:80`



`-p 8080:80`

└─ INSIDE — the port the application listens on. Fixed by the image.
└─ OUTSIDE — the port you type in the browser. Your choice.

Figure 3. Without `-p` there is no route into the container. With `-p`, Docker forwards a host port to a container port.

The left number is yours to choose; the right number is dictated by the software in the image and is stated on its Docker Hub page. So `-p 3000:80`, `-p 8080:80` and `-p 9999:80` all reach the same nginx container — they differ only in what you type in the browser. But `-p 8080:3000` fails, because nothing in an nginx container listens on 3000.

6 Commands

Command	Does
<code>docker run</code>	Create and start a container from an image
<code>docker ps</code>	List running containers (add <code>-a</code> for stopped ones too)
<code>docker images</code>	List images on this machine
<code>docker logs</code>	Show what a container printed
<code>docker exec</code>	Run a command inside a running container
<code>docker stop</code>	Stop a running container
<code>docker rm</code>	Delete a stopped container

Option to run	Effect
<code>-d</code>	Detached — run in background, give the prompt back
<code>-p H:C</code>	Publish host port H to container port C (Section 5)
<code>--name X</code>	Name it X instead of a random name
<code>-it</code>	Attach a terminal — needed for shells and REPLs



A stopped container still exists on disk. It is deleted only by `docker rm`.

The image is untouched throughout — changes made inside a container are lost when it is removed.

Figure 4. A container's life. Stopping is not deleting.

7 Workshop: Guided Practical

Work through these in order. Each line of explanation is short on purpose — the learning is in running the commands and watching what happens.

7.1 Run a web server

```
docker run -d -p 8080:80 --name web nginx
docker ps
```

Open `http://localhost:8080` — the nginx welcome page. You ran a web server without installing one.

7.2 See what `-p` actually does

```
docker run -d --name web2 nginx
docker ps
```

This one is running too, but you cannot reach it — no port was published. Compare the `PORTS` column for the two containers. Then remove it: `docker stop web2 && docker rm web2`

7.3 Read the output

```
docker logs web
docker logs -f web      # follow live; Ctrl+C to stop
```

Reload the browser a few times and watch a line appear per request. `docker logs` is the command you will reach for most often when something breaks.

7.4 Go inside

```
docker exec -it web sh
cat /usr/share/nginx/html/index.html
exit
```

That file is the page you saw. It exists only inside the container.

7.5 A Python container

```
docker run -it python:3.12-slim python
```

A Python interpreter, no installation required. `-it` is essential — without a terminal attached, Python has nothing to read from and exits instantly.

7.6 Python + port publishing together

```
docker run -d -p 8000:8000 --name pyserver \
python:3.12-slim python -m http.server 8000
```

Open `http://localhost:8000`. Everything after the image name replaces the image's default command. Note `8000:8000` matches the port you gave the Python server — mismatch them and the page will not load.

7.7 Clean up

```
docker stop web pyserver
docker rm web pyserver
docker ps -a
```

8 Exercises

No step-by-step instructions here. Each exercise gives you a problem to reproduce; work out why it happens and fix it.

A The container that will not stay alive

PROBLEM

Run `docker run python:3.12-slim python`, then `docker ps -a`.

It already exited. It ran for a fraction of a second and did nothing.

YOUR TASK

Explain why it exited, then run it so the Python prompt stays open and you can type into it.

Hint. A container lives exactly as long as its main process. What does Python do when started with no script and no terminal? See Section 6.

B The page that will not load

PROBLEM

A classmate runs `docker run -d -p 8080:3000 --name broken nginx`.

It starts fine and shows in `docker ps`, but `localhost:8080` gives a connection error. Reproduce it.

YOUR TASK

Find out why, then start it correctly.

Hint. The container is genuinely running, so `nginx` is not at fault. Re-read Figure 3, and recall which port the `nginx` image listens on. Check `docker logs broken` — note that it shows no errors, and think about what that tells you.

C The port that is already taken

PROBLEM

Start one container on 8080, then try to start a second on 8080 as well.

The second fails: the port is already allocated.

YOUR TASK

Explain why, then get both running and reachable at the same time.

Hint. Only one process can hold a given port on a machine — that is the operating system, not Docker. Which side of the mapping are you allowed to change?

D The change that disappears

PROBLEM

Go inside a running `nginx` container and edit the welcome page (no editor — use `echo ... > file`).

Reload the browser: your change is there. Now delete the container, start a fresh one from the same image, and reload again.

YOUR TASK

Explain what happened to your change, and why.

Hint. Look again at Figure 2 and Figure 4. This is exactly the problem Week 3 exists to solve.

E Which tag should you use

PROBLEM

Run `node:20-slim node --version` and `node:latest node --version`.
Compare the versions, and the image sizes in `docker images`.

YOUR TASK

Say which you would use in production, and why.

Hint. What happens to a deployment pinned to `latest` when the project releases a new major version six months from now?

9 When It Goes Wrong

Symptom	Cause and fix
port is already allocated	Something else holds that host port — usually a container you forgot to stop. Find it with <code>docker ps</code> , stop it, or publish on a different host port.
cannot connect to the Docker daemon	Docker Desktop is not running. Start it and wait for it to finish initialising.
Container exits immediately	Its main process ended. Interactive programs need <code>-it</code> . Check <code>docker ps -a</code> for the exit status and <code>docker logs</code> for output.
Running, but browser cannot reach it	No port published, or the right-hand port does not match what the app listens on. Check the PORTS column in <code>docker ps</code> against the image's Docker Hub page.
Changes inside a container vanish	Working as designed. A container's filesystem is rebuilt from the image each time and discarded on delete. Persisting data needs volumes; changing image contents needs a rebuild (Week 3).

10 Before Next Week

Check you can do each of these without looking anything up.

1. State the difference between an image and a container.
2. Explain why a container is unreachable from the browser by default.
3. Given `-p 5000:80`, say which number is the host and which the container — and how you would find the right container port for an unfamiliar image.
4. Start a container, confirm it runs, read its logs, and remove it cleanly.

Reference

- Docker concepts — docs.docker.com/get-started/docker-concepts/
- Publishing ports — docs.docker.com/get-started/docker-concepts/running-containers/publishing-ports/
- Command reference — docs.docker.com/reference/cli/docker/
- Images — hub.docker.com/_/nginx, [_/_python](https://hub.docker.com/_/python), [_/_node](https://hub.docker.com/_/node)